

1. (2.5pts) O que deve ser alterado no algoritmo abaixo para garantir que ele retorne a contagem da quantidade total de componentes conectadas (subgrafos conectados)?

Input : um grafo não-dirigido $G = (V, E)$

```

1  $C_v \leftarrow \text{false} \forall v \in V$ 
2  $x \leftarrow$  selecionar qualquer vértice em  $\{v \in V : C_v = \text{false}\}$ 
3  $(D, A) \leftarrow \text{BuscaEmLargura}(G, x)$ 
4  $c \leftarrow 0$ 
5 foreach  $v \in V$  do
6   if  $A_v = \text{null}$  and  $D_v < \infty$  then
7      $c \leftarrow c + 1$ 
8   end
9 end
10 return  $c$ 

```

1,75

1. A alteração proposta está abaixo. Substituir o que está entre as linhas 1 à 3 por:

```

1.  $C_v \leftarrow \text{false} \forall v \in V$ 
2.  $D_v \leftarrow \infty \forall v \in V$ 
3.  $A_v \leftarrow \text{null} \forall v \in V$ 
4. while  $\exists v \in V : C_v = \text{false}$  do
5.    $x \leftarrow$  um vértice s.t.  $C_v = \text{false}$ 
6.    $\text{BuscaLargura}^*(G, x, C, D, A)$ 
... continua igual

```

* A Busca em largura não inicializa as variáveis C, D e A e as recebe por referência.

2. (2.5pts) O método abaixo foi proposto para determinar se o grafo não-dirigido $G = (V, E)$ tem um caminho Euleriano que não seja um ciclo. O método está correto? Se sim, justifique sua corretude. Se não, informe o que está incorreto e proponha sua correção.

Input : um grafo não-dirigido $G = (V, E)$

```

1 foreach  $v \in V$  do
2   if  $d(v) \bmod 2 \neq 0$  then
3     return false
4   end
5 end
6 return true

```

2,5

2.

2) Entrada: grafo não-dirigido $G = (V, E)$

```

impar  $\leftarrow 0$ 
 $t \leftarrow 0$ 
for each  $v \in V$  do
  if  $d(v) \bmod 2 \neq 0$ 
  |  $\text{impar} \leftarrow \text{impar} + 1$ 
  |  $t \leftarrow v$ 
if  $\text{impar} \neq 2$ 
  | return false
if  $\text{impar} = 0$ 
  | return false
 $(D, A) \leftarrow \text{BuscaEmLargura}(G, t)$ 
foreach  $v \in V$  do
  if  $d(v) > 0$  and  $D_v = \infty$ 
  | return false
return true

```

O método não está correto, ele retorna true se todos os vértices forem par, o que resultaria num ciclo (se for conexo). Além disso, o algoritmo não verifica se é conexo ou não, e para ser um caminho euleriano que não seja ciclo, tem que ter exatamente 2 vértices ímpares.

3. (2.5pts) Para um grafo dirigido e ponderado $G = (V, E, w : E \rightarrow \mathbb{R}^+)$, o que poderia ser alterado no algoritmo de Dijkstra para terminar a execução antecipadamente quando alguma iteração visitar vértices inatingíveis pelo vértice de origem?

3) Entrada: grafo $G = (V, E, w : E \rightarrow \mathbb{R}^+)$, vértice de origem $s \in V$

```

 $D_v \leftarrow \infty \forall v \in V$ 
 $A_v \leftarrow \text{null} \forall v \in V$ 
 $C_v \leftarrow \text{false} \forall v \in V$ 
 $D_s \leftarrow 0$ 
 $(D, A) \leftarrow \text{BuscaEmLargura}(G, s)$ 
while  $\exists v \in V : (C_v = \text{false}) \text{ and } D_v < \infty$ 
  |  $u \leftarrow \text{argmin}_{v \in V} \{D_v : C_v = \text{false}\}$ 
  |  $C_u \leftarrow \text{true}$ 
  | for each  $v \in M(u) : C_v = \text{false}$  do
  | | if  $D_u > D_u + w((u, v))$  then
  | | |  $D_v \leftarrow D_u + w((u, v))$ 
  | | |  $A_v \leftarrow u$ 
return  $(D, A)$ 

```

A ideia central seria realizar uma busca em largura partindo de s , além seria possível saber quais vértices são alcançáveis a partir da origem. Depois, uma das condições da iteração principal seria se o vértice não alcançável por s .

4. (2.5pts) Considere o grafo dirigido $G = (V, E, w : E \rightarrow \mathbb{R}^+)$. Determine a complexidade em tempo computacional para realizar um procedimento que substitua o algoritmo de Floyd-Warshall utilizando o algoritmo de Dijkstra, considerando que a estrutura de dados de Heap mínimo tenha complexidades de tempo $O(1)$ para realizar extração e atualização de chaves. Explique detalhadamente essa nova complexidade de tempo.

4. Para atender o solicitado na questão, precisamos executar a versão de Dijkstra apresentada um n° de vezes igual a $|V|$. Assim, teríamos um caminho mínimo p/ cada par de vértices. Considerando o uso do heap apresentado na questão, cada execução de Dsi. demandaria tempo $O(|E| + |V|) \rightarrow$ executando $|V|$ vezes $\rightarrow O(|E||V| + |V|^2)$